

The 7 Patterns That Cover 80% Of SQL Interviews

1. Aggregate + GROUP BY + HAVING

Use this whenever the question says "total / average / count per X" or "only keep groups where the total is more than...".

- Typical prompts:
 - "Total orders per customer; show only customers with more than 3 orders."
 - "Total revenue per city in 2025."

```
SELECT
  customer_id,
  COUNT(*) AS num_orders,
  SUM(amount) AS total_spent
FROM orders
WHERE order_date >= '2025-01-01'
GROUP BY customer_id
HAVING COUNT(*) > 3;
```

Key idea: **WHERE** filters rows before grouping, **GROUP BY** creates groups, **HAVING** filters those groups using aggregates like **COUNT** or **SUM**.

2. Joins across 2–3 tables

Use this when the question needs columns from multiple tables in one result: "customer name + order amount", "order + product category", "user + event".

- Typical prompts:
 - "Show each customer's total number of orders and total amount spent."
 - "List all customers and, if present, their orders."

```
SELECT
  c.customer_id,
  c.name,
  COUNT(o.order_id) AS total_orders,
  SUM(o.amount) AS total_spent
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;
```

Key idea: choose INNER JOIN if you want only matching rows, LEFT JOIN if you also want "no-order" customers. Always group after the join.

3. Anti-join: "did X" vs "never did X"

Use this when the question says "but never...", "who have not...", "without any...".

- Typical prompts:
 - "Users who signed up but never ordered."
 - "Customers who have not logged in in the last 30 days."

```
-- Users who signed up but never ordered
SELECT u.user_id
FROM users u
LEFT JOIN orders o
  ON u.user_id = o.user_id
WHERE o.user_id IS NULL;
```

```
-- Same logic with NOT EXISTS
SELECT u.user_id
FROM users u
WHERE NOT EXISTS (
  SELECT 1
  FROM orders o
```

```
WHERE o.user_id = u.user_id
);
```

Key idea: join users to actions, then keep rows where the action side is missing.

4. Window functions (latest row, ranking, change)

Use window functions when the question mentions "latest", "top N", or "compare with previous".

a) Latest row per user

- Typical prompts: "latest order per customer", "current subscription status per user".

```
WITH ranked AS (
  SELECT
    user_id,
    status,
    updated_at,
    ROW_NUMBER() OVER (
      PARTITION BY user_id
      ORDER BY updated_at DESC
    ) AS rn
  FROM subscriptions
)
SELECT
  user_id,
  status,
  updated_at
FROM ranked
WHERE rn = 1;
```

Key idea: partition by the key, order by time, keep row 1.

b) Top N (ranking)

- Typical prompts: "top 3 products by revenue", "top 5 customers by spend".

```

WITH product_rev AS (
  SELECT
    product_id,
    SUM(amount) AS revenue
  FROM order_items
  GROUP BY product_id
)
SELECT *
FROM (
  SELECT
    *,
    RANK() OVER (ORDER BY revenue DESC) AS rnk
  FROM product_rev
) t
WHERE rnk <= 3;

```

Key idea: first aggregate, then rank using a window, then filter by rank.

c) Month-over-month change

- Typical prompts: "show revenue per month and the change vs previous month".

```

WITH monthly AS (
  SELECT
    DATE_TRUNC('month', order_date) AS month,
    SUM(amount) AS revenue
  FROM orders
  GROUP BY DATE_TRUNC('month', order_date)
)
SELECT
  month,
  revenue,
  LAG(revenue) OVER (ORDER BY month) AS prev_revenue,
  revenue - LAG(revenue) OVER (ORDER BY month) AS abs_change
FROM monthly;

```

Key idea: aggregate by time first, then use `LAG` to look at the previous row.

5. CASE for conditional metrics

Use CASE when the question wants categories or segments inside one query: "free vs paid", "small/medium/large orders", "active vs churned".

- Typical prompts:
 - "Count how many users are on free vs paid plan."
 - "Group orders into buckets based on amount."

```
-- Free vs paid users
SELECT
  COUNT(*) AS total_users,
  SUM(CASE WHEN plan = 'free' THEN 1 ELSE 0 END) AS free_users,
  SUM(CASE WHEN plan = 'paid' THEN 1 ELSE 0 END) AS paid_users
FROM users;
```

```
-- Bucket orders by size
SELECT
  CASE
    WHEN amount < 500 THEN 'small'
    WHEN amount BETWEEN 500 AND 2000 THEN 'medium'
    ELSE 'large'
  END AS order_bucket,
  COUNT(*) AS num_orders
FROM orders
GROUP BY
  CASE
    WHEN amount < 500 THEN 'small'
    WHEN amount BETWEEN 500 AND 2000 THEN 'medium'
    ELSE 'large'
  END;
```

Key idea: use CASE to turn continuous/complex logic into clean labels, then aggregate.

6. De-duplication and duplicates

Use this when the question mentions "duplicates", "only the latest record", "same email appears more than once".

- Typical prompts:
 - "Find duplicate emails in the users table."
 - "Keep only the latest record per (user_id, product_id)."

```
-- Find duplicate emails
SELECT
  email,
  COUNT(*) AS cnt
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

```
-- Keep latest row per (user_id, product_id)
WITH ranked AS (
  SELECT
    user_id,
    product_id,
    updated_at,
    ROW_NUMBER() OVER (
      PARTITION BY user_id, product_id
      ORDER BY updated_at DESC
    ) AS rn
  FROM wishlist
)
SELECT *
FROM ranked
WHERE rn = 1;
```

Key idea: use GROUP BY + HAVING to locate duplicates, and ROW_NUMBER to pick one row per group.

7. Date filters and time grouping

Use this whenever the question talks about "last N days", "this month", "trend by day/week/month", or "group by weekday".

- Typical prompts:
 - "Show paid orders from the last 30 days."
 - "Monthly revenue for the past year."
 - "Sign-ups by weekday."

```
-- Last 30 days, paid orders
SELECT *
FROM orders
WHERE status = 'PAID'
      AND order_date >= CURRENT_DATE - INTERVAL '30 days';
```

```
-- Revenue by month
SELECT
  DATE_TRUNC('month', order_date) AS month,
  SUM(amount) AS revenue
FROM orders
GROUP BY DATE_TRUNC('month', order_date)
ORDER BY month;
```

```
-- Sign-ups by weekday
SELECT
  TO_CHAR(created_at, 'Day') AS weekday,
  COUNT(*) AS signups
FROM users
GROUP BY TO_CHAR(created_at, 'Day');
```

Key idea: combine date functions to slice time correctly, then reuse GROUP BY, windows, or CASE on top.